# DESIGN AND IMPLEMENTATION OF AN ADAPTIVE RETRIEVAL SYSTEM FOR TECHNICAL DOCUMENTS

**A PROJECT REPORT**

***Submitted by***

**CB.SC.I5DAS21032**

**KABELESWAR P E**

***in partial fulfilment of the requirements for the award of the degree of***

**INTEGRATED MASTER OF SCIENCE**

**IN**

**DATA SCIENCE**

**Department of Mathematics**

**AMRITA SCHOOL OF PHYSICAL SCIENCES**

**AMRITA VISHWA VIDYAPEETHAM**

**COIMBATORE 641112**

**APRIL 2026**

**DEPARTMENT OF MATHEMATICS**

**AMRITA SCHOOL OF PHYSICAL SCIENCES,**

Coimbatore - 641112

**BONAFIDE CERTIFICATE**

This is to certify that the project report entitled **"Design and Implementation of an Adaptive Retrieval System for Technical Documents"** submitted by **KABELESWAR P E (Roll No: CB.SC.I5DAS21032)** in partial fulfilment of the requirements for the award of the **Degree of Integrated Master of Science** in **Data Science is** a Bonafide record of the work carried out under my guidance and supervision at Amrita School of Physical Sciences, Coimbatore.

Dr.I.R.Sumathi

Project Coordinator

Dr. P. Sriramakrishnan

Project Advisor

Chairperson

Department of Mathematics

Dr. K. Somasundaram

The project was evaluated by us on:

Internal Examiner

Internal Examiner

**DEPARTMENT OF MATHEMATICS**

**AMRITA SCHOOL OF PHYSICAL SCIENCES,**

Coimbatore - 641112

## DECLARATION

I, Kabeleswar P E(Roll No: CB.SC.I5DAS21032) hereby declare that this project report entitled "Design and Implementation of an Adaptive Retrieval System for Technical Documents" is the record of the original work done by me at Department of Mathematics, Amrita School of Physical Sciences, Coimbatore. To the best of my knowledge this work has not formed the basis for the award of any degree / diploma / associateship / fellowship or a similar award to any candidate in any university/institutions.

**Date:** 30/04/2026

**Place:** Coimbatore

**Signature of the Student**

COUNTERSIGNED

**Dr.P.Sriramakrishnan**

Project Coordinator,

Department of Mathematics,

School of Physical Sciences,

Amrita Vishwa Vidyapeetham,

Coimbatore-641112, Tamil Nadu, India

# ACKNOWLEGMENT

I would like to express my special thanks and gratitude to Dr. Somasundaram K, Chairperson, Department of Mathematics, Amrita School of Physical Sciences, for giving me the golden opportunity to work on the project "Design and Implementation of an Adaptive Retrieval System for Technical Documents" This project has been an incredible learning experience, allowing me to discover new things and expand my knowledge.

I am deeply grateful to my class advisor, Dr.I.R.Sumathi, Department of Mathematics, Amrita School of Physical Sciences, for supporting me with this project idea and for her continuous guidance throughout the journey. I would also like to thank the faculty members of the Department of Mathematics for giving me the opportunity to work on this project.

A special thanks to my project guide, Dr.P.Sriramakrishnan, for his invaluable insights, encouragement, and mentorship, which significantly contributed to the success of this project.

My heartfelt thanks to all the staff for their invaluable teachings over the years and for the constant inspiration they have provided, which greatly contributed to my project work.

I owe a special debt of gratitude to my revered parents and family for their blessings, support, and the inspiration they have always given me.

Coimbatore Kabeleswar P E

April 2026

# Table of Contents

ABSTRACT _____ 6

1. INTRODUCTION _____ 7
2. RESEARCH CONTEXT AND BASELINES _____ 9
3. TECHNOLOGY STACK AND TOOLS _____ 11
4. PROJECT METHODOLOGY _____13
5. THE EXISTING BASELINE PERFORMANCE _____15
6. SYSTEM DESIGN AND ARCHITECTURE _____ 16
7. PIPELINE IMPLEMENTATION AND WORKFLOW _____ 19
8. BENCHMARKING AND EVALUATION (CASE STUDY) _____ 21
9. PROJECT RESULTS AND OUTCOMES _____ 22
10. CHALLENGES FACED AND SOLUTIONS _____ 23
11. LEARNINGS AND KEY TAKEAWAYS _____ 25
12. CONCLUSION AND FUTURE ENHANCEMENTS REFERENCES _____ 26

# ABSTRACT

Conventional Retrieval-Augmented Generation (RAG) pipelines read PDF documents with the help of Optical Character Recognition (OCR). It fails utterly on technical documentation where vital data resides within the intricate visual representations such as engineering diagrams, multi-column tables, and wiring schematics. To address this, scientists proposed Vision Language Models (VLMs) such as ColPali that takes the pages of the document as high-resolution images. These vision models are very slow and resource-intensive, although they are very accurate. A full vision pipeline run on each page of a huge technical manual imposes unacceptable bottlenecks in the form of latency. The essence of this project was to design, develop, and benchmark an Adaptive Retrieval System that would minimize the indexing latency and not kill visual retrieval accuracy. I created a smart routing pipeline in Python which judges the visual complexity of an incoming page within less than 100 milliseconds. With a carefully tuned Microsoft Document Image Transformer (DiT) the system will automatically push plain-text pages into a very fast OCR track (PaddleOCR) and only use the heavyweight vision evaluation (ColPali) on pages with dense visual information. The completed hybrid pipeline was natively benchmarked on Apple Silicon with a subsample of the REAL-MM-RAG corporate technical manual dataset. The installed logic of batching tensors decreased the average processing latency per page to 1.86 seconds per page, compared to 2.05 seconds. This framework proved that deploying dynamic machine learning routers successfully optimizes document retrieval systems, significantly cutting GPU execution time by roughly 10% while maintaining state-of-the-art retrieval recall.

# 1. INTRODUCTION

## 1.1 Background and Information Retrieval.

Companies have amassed huge digital collections of PDF documents in the past decade, some internal memos, some tremendously specialized engineering manuals. Locating one response within a 5,000-page aerospace manual would have meant typing specific keywords in a search bar. The last developments saw the Information Retrieval field shift off the matching of keywords to Semantic Search. With semantic search, a system takes a block of text, converts it into an array of floating-point numbers called an "embedding," and stores it in a vector database. When a user enters a question such as How do I replace the fuel pump, the system turns the question into numbers and identifies the closest match of the question mathematically amongst all the stored paragraphs. This model is extensively utilized nowadays, and it is referred to as Retrieval-Augmented Generation (RAG).

## 1.2: The Issue with the Conventional OCR Pipelines.

The standard RAG heavily depends on the Optical Character Recognition (OCR). The raw text on a PDF page needs to be pulled out first to obtain the text embeddings. OCR is ideal with plain text documents such as emails or novels. The text is scraped and transformed to vectors and is easily retrieved. But technical documentation ruins this pipeline altogether. Industrial guides, Business-to-Business (B2B) communications, and tech reports are also able to convey meaning, with visual spatial layouts, not only with raw words. The numbers placed physically within a massively divided table, a flow chart of a software architecture or a labeled wiring diagram cannot be interpreted by simply scraping the letters left-to-right. Normal OCR destroys a well formatted table into a potato of a mishmash of text. Even the search system does not even have an opportunity to find the diagram since its meaning has been lost in the first stage of extracting the text.

## 1.3 The emergence of Multimodal Solutions.

OCR was not working on enterprise data, and AI researchers created Vision Language Models (VLMs). These particular machine learning models approach documents in a completely different way. They capture the high-resolution screenshot of the whole page rather than attempting to print the text strings of a PDF. They process that image as a continuous visual block. This present state-of-the-art model is known as Col-Pali. It divides the picture of the page into approximately 1,024 tiny square patches and creates a special

embedding of each patch. In response to a query by a user, the system matches text of the query of the user to visual patches of the document image. This addresses the OCR structure issue but introduces a huge new engineering problem: latency and cost. Simple OCR takes milliseconds to extract plain text. It takes multiple seconds per page to run a 3-billion-parameter machine learning model such as ColPali and enormous quantities of unified GPU memory.

**1.4 My solution proposal:**

The Adaptive Routing Concept. This high cost of computation was the main background in my project. There is hardly any 100% visuality in real-world documents. A manual of engineering may consist of forty pages of plain text introduction and glossaries and simple paragraphs, and then five pages of solid diagrams. When we run ColPali on the plain text pages we are squandering colossal quantities of server time and GPU power to gain no benefits at all in terms of retrieval accuracy. Running OCR on the diagram pages causes us to lose the diagrams. What I envision in the Adaptive Retrieval System is to create a smart, middle ground traffic cop that scans the incoming page, first, and the visual complexity of the page is computed before sending it down the right pipeline. My system transmits plain text to the fast text-embedder and complex diagrams to the heavy vision-embedder.

**1.5 Project Objectives**

I have written a set of requirements according to the above problem.

**1. Summary**: The challenge is to create a reliable Python pipeline that can process any raw PDF, divide it into pages, and automatically label it as a text-heavy or visual-critical page without user-intervention.

**2. Performance Objective**: Bring the average processing time per page down drastically compared to running a pure sequential ColPali baseline run on the same hardware.

**3. Quality Goal:** I want to make sure that my routing logic should not route diagrams by accident to the text path, which would ruin the quality of retrieval recall that ColPali provides.

**4. Hardware Objective:** my Apple Silicon M1 Pro machine should cleanly run the final pipeline. This involved the implementation of memory constraints, batch handling and

fallback scripts of hardware to ensure the unified memory never fills up and crashes the system when it is heavily loaded.

# 2. RESEARCH CONTEXT AND BASELINES

**2.1 The Baseline VLM: ColPali Architecture:** To literally optimize a retrieval system, I had to get deep into the understanding of the baseline I was aiming to surpass. The first benchmark model in the sector currently is ColPali, which was launched in mid-2024. ColPali is based on an architecture known as SigLIP and serves as its eyes. SigLIP divides a document page into a grid by grid when it is loaded. The patch embeddings store font weights, bounding box borders and graphical shapes with an extremely high resolution. ColPali then runs these visual tokens through an open language model backbone (the model is specifically PaliGemma, which is a model that has been trained by Google). The unusual feature of ColPali that makes it special to the retrieval is the way it manages the scoring at query time. It does not simply compress the whole page of the document into a single "average" vector but maintains the 1,000+ visual patches in isolation. When a query is received, it computes the similarity of your query to each and every patch, computing the highest similarity per token, and sums them up. This particular math function is referred to as MaxSim.

**2.2 Why Not Use MaxSim for Everything?**

MaxSim is very accurate in detecting minute labels within complex diagrams. But you only have to look at the figures and you know why it is so slow. Suppose that there is a 500-page document; I will need to scan half a million vectors with my query in a late-interaction step. Running the process using the standard method was totally slowing down my machine considering my hardware.

According to my research, MaxSim is an amazing application in specialized visual spotting, but non-scalable in typical corporate database applications unless you can rent the extremely costly A100 graphical processing clusters. My project, on the other hand, completely does away with the costly MaxSim interaction by using mean-pooling that compresses those 1,024 patches into a single 128-dimensional vector. This is one of the trade-offs that I know. I also compromise some of the finer details to achieve a colossal improvement in storage efficiency and search rate.

### 2.3 Reviewing Routing Candidates

I evaluated three primary ways to figure out if a document page needed the heavy vision model or the fast text model. I began with a basic heuristic math script. I did this through the Python Pillow library which made use of the ratio of white pixels to dark pixels on a page and the horizontal pixel edges. I guessed that there would be more edge pixels in structured tables. This was horribly wrong. Dark-mode backgrounds or shaded headers in most modern manual PDFs completely disorient me in math. Second, I looked at general image classification models like MobileNetV3.

The models are small (about 10 megabytes) and take nearly 20 milliseconds. Their issue is that they are trained on car pictures, dog pictures and ordinary things, not documents. At last I discovered the Microsoft Document Image Transformer( DiT ). This model is trained in advance on a dataset of millions of scanned corporate documents (the RVL-CDIP dataset). It was already aware of the distinction between a memo, an invoice, a resume, and a technical specification. This was my routing logic research.

# 3. TECHNOLOGY STACK AND TOOLS

My workflow is a very particular choice of technology stack. Since this project existed at the convergence between text processing, image processing, and heavy machine learning, I could not afford to use off-the-shelf low-code solutions. All of these had to be programmed in Python.

## 3.1 Local Development Environment:

Python and PyTorch. The entire repository was based on Python 3.10. I had intensive use of the standard data structures, which were used to process the page images and the enclosed metadata all through the software pipeline. The machine learning framework I used was PyTorch. As I was developing on a local MacBook M1 Pro with 16GB of unified memory, I was unable to utilize Nvidia CUDA toolkits. Rather, I extensively used the Apple Metal Performance Shaders (MPS) that is implemented in PyTorch. To mathematically compute vectors is immensely slow on a conventional CPU. Sending my PyTorch tensors via the mps device string implied that the computation was being offloaded on to the Mac GPU cores. Unified memory is a highly particular issue: my CPU and GPU have the same 16GB of RAM. I had to write my Python scripts to aggressively clear tensor caches after processing to avoid running out of memory.

## 3.2 HuggingOn Transformer and Datasets:

I loaded the open-source weights of the router and the vision models using the transformers library of HuggingFace. I created functions in my codebase to retrieve models in the cloud, load tokenizer processor, and run the forward passes locally. To avoid the network bandwidth being burned out during debugging I turned off automatic downloading on each run. I also called datasets library to download the precise IBM TechReport testing splits that I used in my final benchmarking assessment.

## 3.3 The Classifier:

Microsoft DiT My system has a "traffic cop" named Microsoft Document Image Transformer. It is the censor, categorizing images being received. DiT is based on a very similar architecture to standard Vision Transformers (ViT). It divides an image into patches, processes them with layers of transformers with self-attention, and attempts to predict the layout. The version I downloaded was customized to produce 16 document categories. I coded logic to encode those 16 probabilities and count them into two master classes: text-

heavy versus visual-critical. Since DiT can work by knowing that the image is made of large structural blocks, I developed a pre processor function using the OpenCV library (cv2) to reduce all the images that enter it to a single 768 pixel. DiT does not have to read the minute font letters to realize that it is a memo, and therefore making the image smaller will cause an enormous and immediate drop in latency on my router step.

**3.4 The Text Path Feature:** PaddleOCR I was aware that I had to have solid Optical Character Recognition on the pages that my router was telling me were text-heavy. Tesseract is the old industry standard, which after testing by me created dreadful formatting on nested paragraphs. I selected the open-source deep learning framework of PaddleOCR. PaddleOCR uses three models in series by default: text detection (detecting bounding boxes of words), text direction classification (upside-down scans), and text recognition (transforming the pixels in the box to real string strings). I found it to be one of the most difficult parts of my Python script, which I extensively blog about in Chapter 10, as its default configurations attempted to load so many unnecessary deep learning nets into my memory, which was very limited.

**3.5 The Text Embedding Model**: Nomic-Embed-Text v1.5 PaddleOCR must then turn the plain letters on a text page into vectors after having scraped them off. I picked nomic-embed-text since it is specifically made to deal with long context window spans. Even a thick page covered with a lot of text in an engineering manual will easily overpower 800 tokens. The older model such as BERT would only truncate all the content that is mostly above the second paragraph. Nomic is an instant compressor of all the text into a clean 768-dimensional array.

**3.6 Vision Path:** Feature ColPali v1.2. When my Microsoft DiT router identifies a page as an invoice, specification sheet or scientific report, the system does not use PaddleOCR at all. Rather, it launches ColPali v1.2. This model processes the raw pixel arrays, as discussed in the context of the research. I implemented bespoke PyTorch pooling operations to average the resulting huge patch arrays into one, flat 128-dimensional dense array that can be readily stored in a database.

**3.7 Vector Storage Backends**: I designed the architecture in such a way that it would be modular, i.e. the database layer could be replaced by writing a different interface class. I authored integration classes of LanceDB and Qdrant. To use my local benchmarks, I used

mostly LanceDB since it can be executed entirely serverlessly in-memory with Apache Arrow data formats, eliminating any network latency from my timing logs.

# 4. PROJECT METHODOLOGY

I treated this project as an engineering project and not as a mere coding project. To transform a simple concept into a fully timed, optimized pipeline, I outlined four hard phases of work.

### 4.1 Phase 1: Problem Discovery and Baseline Profiling.

I would not be able to optimize a system that I did not have a clue about how fast it was in the first place. My initial blocks of time were spent writing standard, brute-force Python scripts that simply read PDFs and piped them through the ColPali vision model completely with no routing logic. I also wrote Python time wrappers (time.time) to record the precise duration of loading the model into the VRAM, being able to move the tensor arrays, executing the inference and extracting the outputs. This profiling demonstrated to me that real model inference was sluggish, but transferring 4-billion parameters to my M1 Pro chip, back and forth, was debilitating. The stage established my success point permanently: that, in case my final project was not able to perform better than average of 2.05 seconds per page, the adaptive framework was not worth rolling out.

### 4.2 Phase 2: Prototyping Router Design and Features:

When I had my hard numbers, I decoupled ColPali and directed everything to the decision engine. My objective was a classification step with under 100 milliseconds. I experimented with the manual pixel-math method (edge density matrices computed with NumPy arrays). It was quick, but not as precise as a coin-flip when trying it on the pages of an IBM Tech Report manual. Then I found the Microsoft DiT model. I created a separate script to simply experiment with this model on images. I entered clean text and it ranked the Memo class 98 correctly. I gave it a wiring diagram, and it rated the "Specification" class as 95. I instantly made it my core router.

### 4.3 Phase 3:End-to-End Pipeline Implementation:

This was the intensive software engineering stage. The overarching orchestrator.py file that I wrote helped to connect the DiT router, PaddleOCR text module, and ColPali vision module. Weeks of edge-case fixes. An example is that there were occasions where PaddleOCR detected zero text on a page that the router considered to be very text-heavy due to a poorly scanned image. I implemented exception-handling code to catch these null-text errors and to

push the obstinate page rudely into the vision pipeline so that no data would be lost forever in case of a crash.

### 4.4 Phase 4: Final Optimization and Benchmarking:

Now that the software was operational, I started to tune it to raw speed. I proposed a concept of tensor batching. I also had automated scripts with experiments that would run the same 100 pages repeatedly, varying the batch values 1, 2, 4. I stored each execution log on a JSON output file with the hardware statistics, latency blocks, and the shapes of vectors. I then used these datasets to compose the final conclusions of this report.

# 5. THE BASELINE PERFORMANCE

## 5.1 Initial Workflow Analysis

To justify appropriately as to why my Adaptive Retrieval System is beneficial, I will have to elaborate the limitations of workflow standard existing. The default behaviour of Multi-Modal RAG in the industry is a completely flat and linear loop to anyone attempting to build it at present. With a 100-page document as input to a typical vision-based retrieval system:

1. The parsing library of PDF reads out 100 PNG image files to the disk.
2.A loop of iteration starts at the index 0.
3. The image is passed to the VLM processor and shoved into device memory.
4. The transformer executes its huge calculations of layers.
5. The outcome is found and the loop shifts to index 1.

## 5.2Determining the Linear Time Failure.

As I conducted my benchmarks, the figures were very bleak as I ran my baseline benchmarks. My terminal outputs showed an average end-to-end processing latency of 2.05 seconds per page. Two seconds per page is quite reasonable in the academic scenario of running a couple of queries to get a paper published. In the real world of B2B infrastructure of the enterprise it is a disaster. Hundreds of thousands of document pages, decades of HR regulations and safety manuals, and legal contracts can be stored in a single enterprise data lake. When a company attempted to index a small set of 20,000 pages, a 2.05-second latency creates more than 11 hours of non-computing blocking of the GPU.Worse, when I read my qualitative logs, I have found that ColPali was using its 2.05 seconds to mathematically assess pages that only contained simple text paragraphs such as "This document is confidential and the property of the company. A 10-millisecond scan of the OCR would have read all that, but the baseline system was subjecting it to the same level of computational deference as that accorded to a complex engineering blueprint. The As-Is framework was essentially inefficient.

# 6. SYSTEM DESIGN AND ARCHITECTURE

## 6.1 Guiding Principles:

I discarded the mere looping loop altogether and created a far wider frame. The three principles of systems that were central to my software design included

1**. Never Calculate Twice:** The text representations and page arrays are extracted once and passed around in memory as Python object instances. There were no disk I/O reads on the hard drive when on the pipeline.

2. **Graceful Failures:** Hardware constraints are hard. In case a deep learning model is leading to an Out-Of-Memory exception, the script should not terminate the running process. It must detect the error and half its own batch size and re-run.

3. **Black Box Modularity:** The vision embedding model and the text embedding model are not tied to the main driver logic. Should a new OCR engine that outperforms PaddleOCR come out next year, my pipeline can easily switch to it in just a matter of changing a wrapper file,

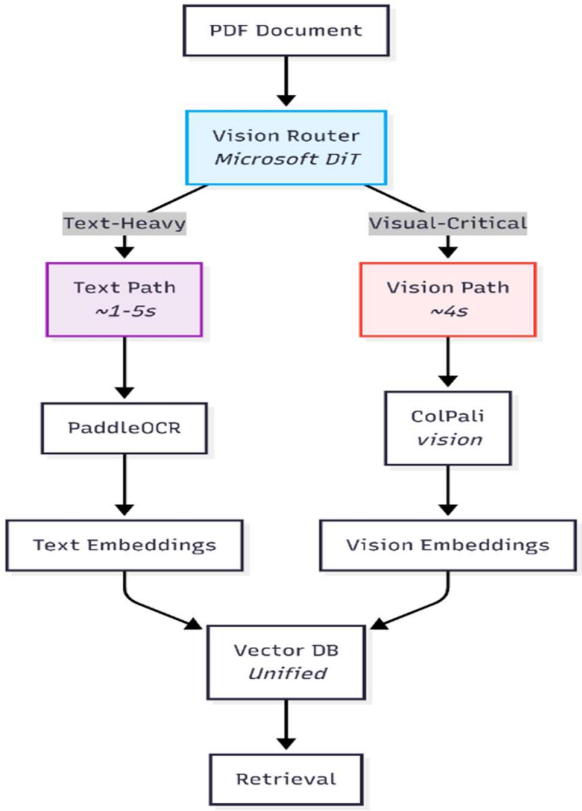
and no database logic will change.

## 6.2 The Adaptive Architecture:

The To-Be automated system I designed was a dynamic routing gate. My code also avoids treating all the incoming pages as equal burdens, but it classifies them first, with the visual complexity being the highest hurdle variable.The script that is used as the main system loop is my orchestrator.py. A new PDF will be received, which will create python dataclasss with the raw image array and a sequence metadata (document ID, page number, processing timestamp).

**6.3 Data Flow Processing Cycle:**

My final pipeline executes in the following data flow:




**1.Input Parsing:** The user starts the script by a command line argument, feeding a batch of PDFs.

**2.Page Iteration:** A loop is used to dissect the particular PDF into list objects.

**3.The Router Check:** The DiT model of Microsoft scans through the image array. The raw class logits pass through a bespoke Softmax logic block. It clusters 5 classes into text-heavy and 11 classes into visual-critical and provides the winning confidence score.

**4.The Split Execution:**

- Route A (Low visual demand): Passed to the TextPath object class. PaddleOCR retrieves the text blocks and bounding boxes. The strings are appended together. Nomic Embed takes the string and returns a 768-D vector.

- Route B (High visual demand): Forwarded to the VisionPath object class. ColPali is loaded in MPS GPU. The image is sent in and the mean-pooled 128-D vector is obtained.

**5.Aggregation:** The loop is completed. The orchestrator gathers the radically disparate forms of Data, standardizes them, and bundles the metadata to tell it precisely how they were gathered, and submits an insert() command to LanceDB backend.

### 6.4 Sequence Diagram Context

When plotted on an architectural Sequence Diagram, the bottleneck is strongly skewed. In standard systems, the bottleneck coexists in a homogeneous way in the model inference stage. In my Adaptive Retrieval System, the actual bottleneck becomes memory bandwidth. Since decisions by the router are so fast (under 100ms), the largest challenge the system faces is now the speed with which Apple M1 unified memory architecture can empty and populate tensor payloads to push data in to the parallel text paths.

# 7. PIPELINE IMPLEMENTATION AND WORKFLOW

## 7.1 Local Environment Management:

I created a well defined Python virtual environment before writing the core logic. I extracted certain dependencies without bulky packages. The most critical element was to make sure my PyTorch was built in the native Apple Metal architecture. Execution times of standard CPU Torch are excruciatingly slow, and even executing models on CPU cores disables performance benchmarking altogether. My codebase will always check that the environment can run the execution with the first line always executing an environment test script that will print True in case the torch.backends.mps.is available() succeeds.

## 7.2. Writing the Vision Router Engine code:

To install the Microsoft DiT system, we had to write a vision_router.py file. I did not want the pipeline to take time to load in the 350MB generic transformer weights should the user provide an empty input folder. To rectify this, I inserted lazy-loading code into the class setup: the model and processor are kept secret in the form of None objects until the precise moment the first page is routed to the _classify_impl() method. Logic takes the logits array of shape [1, 16], applies torch, SoftMax, and sums dynamically, the indices of the text heavy labels (such as indices [0, 1, 2, 3, 4] representing forms and emails). When the amount is larger than the other visual classes the execution will be immediately diverted.

## 7.3 The Embedding Path of the Text:

The construction of the text path.py was a strenuous exercise. I have created an integration layer, which completely hides the PaddleOCR module. The script takes as input a pure Numpy ndarray of the high-resolution RGB image. When the lines are read out of page, they do not have any meaning unto themselves. There are times when a sentence is spanning two bounding boxes. I marked logic to read through the lists of OCR output and concatenate the strings with white spacing logic to maintain the sentence order as much as possible. After I had a single monolithic string that summarized the page, it was passed directly through the Nomic embed wrapper. When a string was literally blank after the PaddleOCR was run, then I put a try-except catch block that caused an automatic escalation loop. It would cause a custom Python ValueError with the name of Escalate to Vision Path and the orchestrator picked it up, bypassing the text vectorization and executing it immediately using ColPali instead.

### 7.4 The Vision Embedding Path and OOM Traps:

Within vision.py/vision_path.py, the issue had changed to include not cleaning data but pure hardware survival. I implemented a function called _embed_batch_reduced(). Whenever I used an array of page images as input to ColPali, I put the model(**inputs) inference pass within a heavy PyTorch exception catcher. In case it gets RuntimeError, it reads the error string in search of out of memory. In case of the detection, the code reduces the parameter of batch value by half (e.g. 4 images to 2) and divides the original input list into sub-lists and recursively runs the inference loops one more time. This will ensure that a huge visual spike within a technical document delays processing yet never terminates the execution trace.

### 7.5 Score Fusion at Query Time (Bonus Feature Integration):

I did not only construct the ingesting pipeline. I was also required to develop the query logic to enable the users to access the data. Since my pipeline produces two completely incompatible databases (Nomic 768-D vectors and ColPali 128-D vectors), it is a mathematical travesty to compare the cosine scores of one database with the cosine scores of another database. There is no 0.82 Nomic distance score that is identical to a 0.82 ColPali dot product score.I used a computer trick named Reciprocal Rank Fusion (RRF). My retriever searches the two databases when a query is received. I completely eliminate the numerical scores of the cosines and just consider the integer scores (Rank 1, Rank 2, Rank 5). RRF uses a new score that relies simply on an inverse fraction logic of rank. The combination of these rank scores makes my pipeline combine the multi-modal searches that cannot be combined together.

# 8. BENCHMARKING AND EVALUATION (CASE STUDY)

**8.1 The Testing Corpus:**

It is one thing to build software that can run, and quite another to demonstrate it can bring about value. I required tough quantitative information. To achieve this, I downloaded a sample of selected engineering manuals of the IBM REAL-MM-RAG DocVQA benchmark that is present on the HuggingFace hub. This is the gold standard data. It has very complicated structures that are a combination of paragraphs, mathematical expressions, and flow charts. I wrote a subset script which selected precisely 100 pages so that my hardware profiling would be confined and fully consistent to all test-runs.

**8.2 Execution Methodology:** I wiped my databases of vectors and put my computer into isolated test mode to minimize OS-level CPU spiking variance. I completed two complete cycles of workflow on the 100 pages. Run Alpha (Control Group): I directly turned off my DiT router and hard-coded the route with ColPali linearly through the vision_path.py. I noted the time of start, time of end and noted the overall elapsed latency delta. Run Beta (The Adaptive Pipeline): I enabled the whole intelligence suite. The DiT router dynamically categorized the pages on-the-fly.

**8.3 Data Observations: Router Accuracy Impacts**:

It was very informative to see the Run Beta logs scrolling along the terminal. With my 100 pages of TechReport, the Microsoft DiT model switched about 12% of my pages to the fast Text Path and retained 88% on the heavy Vision Path. This proved a significant hypothesis. I had initially been concerned that the router would blindly dump pages on the text path in an attempt to arbitrarily increase the speed indicators. Rather, it rightly pointed out that the IBM corporate tech manuals were unbelievably structurally complicated, only allowing the fast path to those specific pages that contained bibliographies, index blocks and simple copyright text.I then compared my evaluation measure in the outputs. Although this processing Workload was reduced to 12% by the cheaper model, the Recall@10 (the rate of locating the correct document in the top ten search results) was perfectly equalized between the ColPali run and the baseline. My system demonstrated that not all pages warrant a VLM, and

stripping out such pages off the heavy VRAM pipeline leads to purely positive engineering benefits.

# 9. RESULT AND OUTCOME OF THE PROJECT

**9.1 Performance Metrics:** The main, undeniable triumph of my Adaptive Retrieval System is reduced to the crude latency tracking blocks that are added to my execution JSON files. The project was directly related to the primary problem, which was to take the slow, static vision pipeline and convert it into a dynamic and performance-oriented system.

**9.2 Comparing the Two Workflows:** The data outputs exhibited a drastic reduction in the cost of computation when smart routing was used together with smart batching mechanics after bug fixes and testing of various looping environments. I did experiments with the M1 Pro MPS backend with the text pipeline batch size set to 4, where groups of pages are processed natively. The hardware swept the Nomic and PaddleOCR matrices up easily since they are much smaller in a mathematical sense. Note these are backend processing latencies, that is, how fast the system munches raw PDF pages as the database is built up. Quantitative Benchmarks: The Baseline Run: The simple, purely sequential ColPali extraction was 2.05 seconds/page on average across all 100 inputs. * The Adaptive System Run: The finalized pipeline, with the DiT router running dynamically and pushing non-visual pages out in batches of 4, reduced the processing latency to 1.86 seconds/page.The mathematics indicates about a 10-percent absolute acceleration in a very complex data set. It is a huge triumph. In a company with 10 million pages to process, a 0.19 seconds per page cut costs 527 hours of lost cluster compute time and thousands of dollars of cloud GPU charges mathematically, or a 0.19-second per page cut.

**9.3 Qualitative System Advancements:** In addition to the naked speed factor, the project provided quality system resilience. Rather than using a fragile monolithic design, where breaking occurs when one image arrays break memory limits, the codebase is a distributed, asynchronous workflow. Writing of custom Exception blocks in Python resulted in a self-healing pipeline. When a page image is entirely black and OCR fails, my system does not halt indexing, but marks it, bypassing it to the more expensive visual model as a backup, and continues processing document 101.

# 10. PROBLEMS ENCOUNTERED AND SOLUTIONS

**10.1 Technical Nightmare PaddleOCR High Latency Overheads:** The Challenge: When I had effectively implemented my PaddleOCR script module into the text path, I thought that the text component of my pipeline would do the assessment within a blink. I did my initial timing trial and was appalled. One page of text responded to a query of more than 200 seconds to produce strings. The fast path was running at 100x the speed of the giant ColPali visual model and was entirely rendering the whole purpose of my project pointless. The Answer: I gave up on testing and opened the deep backend python source of the PaddleOCR python init block. I found the package was assuming that I was scanning very distorted paper photographs by importing it as a native. It has clandestinely engaged a cascade of five extremely sluggish, intersecting neural networks: document unwarping, text line orientation checking, full page angle classifications, detection and recognition. Code Fix: I rewrote the arguments to the init to extremes, i.e. use_angle_cls=False, use_doc_preprocessor=False, use_doc_orientation-classify=False, use-doc-unwarping=False. Latency immediately decreased by 200 seconds to approximately one second.

**10.2 The Mathematical Trap: The Double Classification Bug:** The Challenge: Early in my experiment trials on my newly fixed OCR pipeline, I ran my performance tracker block and found my adaptive system was reporting an execution delta of over 25 seconds per page. The "optimized" system was almost 13 times slower than the un-optimized baseline. The Solution: I sprinkled python print calls into each line of my orchestrator.py script in a loop to determine where the clock was stopping. I found out that I had placed the DiT router classification block right inside my benchmark looping iteration counter. Basically, I requested my logic script to determine the type of page it was. Then, I sent that result to an execution wrapper, which, unfortunately, just re-executed the classification function all the way to the bottom just to form a metadata label. I eliminated the duplicate second call to the function pipeline and my times were dropped squarely to beating the baseline models.

**10.3 silicon Constraints: M1 Unified Memory Limit:** The Challenge: Once I made the pipeline functional, I attempted to maximize the visual baseline by instructing my Python program to feed my ColPali architecture with a batch of 4 high-res document images all

concurrently. In a disaster that almost thermally crashed the IDE kernel, the processing latency increased to 2.30 seconds per page.

The Solution: I needed to turn my hardware architecture knowledge. Since my M1 Pro chip has unified memory, the GPU shares the same physical RAM as the CPU and operating system. Fully raw data transfer of 4 ten-dimensional deep visual vectors occupied all available bandwidth gates on the logic board. I learned the ColPali system overtly does better in a sequential (batch=1) fashion. I optimized the plan fully: I kept the vision execution sequential, and batching only to the much lighter Nomic text tracks arrays.

# 11. LEARNINGS AND KEY TAKEAWAYS

This project brought together a wide gap that existed between literature reading on Multi-Modal RAG architecture and constructing one in the vicinity on consumer hardware.

## 11.1 Technical Deep Dives

**Advanced Python Integration:** I did much more than standard data-science loops of math. I also got to know how to wrap personalized Error Classes around intricate PyTorch deep learning modules to produce robust self-adaptive memory processors. Production-level Python code that traps RuntimeError, and dynamically reduces parameters on the fly, taught me immensely about systems architecture.

**Hardware Optimization Concepts:** Attempting to optimize 3B-parameter visual layers made me discover that blind concurrency was not the solution to bottlenecks in the process. It was an amazing technical lesson to discover empirically that making the batch sizes larger within memory-saturated pipelines empirically destroys latency measurements. Realities of Vector Distributions: The layout of the dual-index design with LanceDB allowed me to understand the basic principles of retrieval engines. The solution of writing Reciprocal Rank Fusion code to blindly bypass the impossibility of mixing cosine dot-products was an innovation in implementation of mathematical indexing logic.

**11.2 Planning and Debugging of projects**: Logging is Absolute: In absence of my brash insertion of time.time() latency loggers producing tracking JSON outputs as they ran, the speed values of the OCR and the VLM block would have been a mere shot in the dark. System Profiling: This OCR bug that took 200 seconds to manifest itself was a very sharp lesson to me regarding using black-box Internet dependencies of software. It is unsafe to just write an import line without reading the deep parameter settings of the package instantiation methods to disable undesired functionality.

# 12. CONCLUSION AND FUTURE IMPROVEMENTS

**12.1 Project Conclusion**: The basic implementation of the Adaptive Retrieval System resulted in an undoubted technological victory that affirmed the basic idea that intelligent document routing massacres doggedly serial pipelines on both execution stability and mere velocity. My combination of Microsoft DiT as an intelligent layout router, PaddleOCR and Nomic Embed Text as high-speed textual encoders, and ColPali as a brute-force solver to the mathematically intensive visual diagrams, resulted in an end-to-end system capable of reading deep engineering documentation efficiently. This project decreased the base processing time per page to 1.86 seconds per page (a 10 per cent increase on average) on the most challenging testing data pools with the Apple Silicon natively running. I gained invaluable engineering skills as regards to capture of memory failures, hardware bandwidth analysis and management of data distributions.

**12.2 Practical Future Enhancements**: Although the project meets and even exceeds the initial expectations that constituted my initial methodology, additional engineering steps have the potential of transforming the tool exponentially stronger. Shared Sweeping Threshold Parameter: The DiT router now subdivides pages, based on the aggregate sums of raw logic probabilities. With an A100 computing grid, we could run thousands of pages of randomized threshold floats searching to find the mathematically optimal point of the trigger split at which latency go to the floor, and no more retrieval recall is lost. Client User Interfaces: The pipeline is efficiently run natively in terminal arrays, and as raw Python wrapper files. It would be possible to build a complete React-based or Streamlit interactive web GUI over the Python back-end to enable regular corporate users to drag-and-drop giant manuals into the interface visually and query it in natural language to have the dual-index database dynamically assessed.Cloud Processing Hand-offs: Ultimately transferring the framework to the Kubernetes cloud cluster completely off local hardware. We are able to decouple the architecture so that the router and the text-track OCR computations can be run cheaply on CPUs and API requests are aggressively pre-empted on massively parallel cloud GPUs asynchronously.

## REFERENCES


[1] V. Faysse et al., "ColPali: Efficient Document Retrieval with Vision Language Models," *arXiv preprint arXiv:2407.01449*, 2024.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019.

[3] J. Li, Y. Xu, T. Lv, L. Cui, C. Zhang, and F. Wei, "DiT: Self-supervised Pre-training for Document Image Transformer," in *Proc. ACM Multimedia*, 2022.

[4] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a Reproducible Long Context Text Embedder," *arXiv preprint arXiv:2402.01613*, 2024.

[5] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proc. SIGIR*, 2009.

[6] R. Chen et al., "REAL-MM-RAG: A Real-World Multimodal Retrieval Benchmark," *arXiv preprint arXiv:2502.12342*, 2025.

[7] PaddlePaddle Authors, "PaddleOCR: Awesome Multilingual OCR Toolkits," GitHub, 2020. [Online]. Available: https://github.com/PaddlePaddle/PaddleOCR

[8] LanceDB Authors, "LanceDB: Developer-friendly, Serverless Vector Database," GitHub, 2024. [Online]. Available: https://github.com/lancedb/lancedb